Swinburne University of Technology

School of Science, Computing and Engineering Technologies

COS30019 – Assignment 2A

Due Date: 19th April 2026

Tutor: Hy Nguyen

Tree-Based Search

Student Name	Student ID	Contributions	Signature
Meg Howells	105352120	Introduction, Insights, DFS implementation, DFS features, ALT testing (T9-T13), Research.	
Steve Shandran Henry Albert	105401284	BFS Implementation, BFS features, ALT implementation, Conclusion	
Chi Thien Ly	105223103	A* implementation, A* features, A* testing (T6-7), DLS implementation	
Sunderiya Buyannemekh	104066792	search.py helpers, combining algorithms, GBFS implementation, GBFS and DLS reporting T1-4, T8, T14, T15	

1. Table of Contents

1. Instructions **3**

2. Introduction: **3**

3. Features **4**

DFS: *4*

BFS: *4*

GBFS: *4*

A Star: *4*

CUS1 – Depth Limited Search: *4*

CUS2 - ALT: *5*

Bugs: **5**

4. Testing: **5**

5. Insights: **8**

6. Research: **10**

ALT: 10

7. Conclusion **11**

Acknowledgements/Resources **12**

References **13**

1. Instructions

Executing search.py:

The main search.py has 6 different algorithms to solve route-finding problem. For execution follow the below:

- Navigate to the folder containing the search.py using command line interface.
- Run "**python search.py <filename> <method>" or python3 search.py <filename> <method>"** (Example: python search.py test/t1_gen_correct.txt BFS)
    - Replace the <filename> with a valid path to your test file
    - Choose from **BFS**, **DFS**, **GBFS**, **AS** and **ALT** to replace <method> of the command below
- For method **DLS**: run "**python search.py <filename> <method> <limit>"** or **python3 search.py <filename> <method>"** (enter an integer limit after the method. (Example: python search.py test/t1_gen_correct.txt DLS 3)

Understanding results:

The program will return outputs following this structure:

filename method	test/t1_gen_correct.txt BFS
goal number of nodes created	5 5
path	1 2 5

2. Introduction:

The Route-Finding Problem involves a single agent finding the lowest cost path between an origin and one or more destination nodes on a weighted 2D graph [Aitchison, 2012]. The nodes are expressed as coordinates (x, y) and are connected by edges each carrying an associated cost. Edges may or may not be directed, permitting one-directional traversal only, or undirected, permitting bidirectional travel.

A graph is a data structure that consists of a set of vertices (nodes) and a set of connecting edges. The connections between nodes are stored in an adjacency structure to allow efficient lookup of a given node's neighbours. A search tree is a structure that is generated in the search process, distinct from the underlying graph. It represents the paths explored by the agent from the root node outward. The root node, also referred to as the origin, has child nodes that serve as reachable neighbours. The frontier refers to the collection of nodes that have been discovered but not yet expanded. A visited set is maintained to prevent the agent revisiting already discovered nodes, furthermore, avoiding infinite loops between nodes [GeeksForGeeks, 2025].

In order to solve the Route-Finding Problem, six tree-based search algorithms have been implemented. These are divided into uninformed and informed methods. Uninformed methods have no knowledge of the destination node's location or cost and explore the graph systematically these include *Depth First Search (DFS)* and *Breadth First Search (BFS*). Informed methods employ a heuristic, in this case the straight-line distance from a current node to the destination node to guide the direction of travel *Greedy Best First Search (GBFS)*. Plus, *A* ("A star") that uses distance from origin to goal node with the distance from the current node.* Additionally, two custom algorithms have been implemented: uninformed *Depth Limited Search (DLS)* and informed *A* Landmarks Triangle Inequality Search (ALT).*

The ALT search is an informed method with the aim of finding the optimal path with the least number of moves. Each algorithm manages the frontier differently, determining the order in which nodes are explored, thereby resulting in varying levels of optimality, completeness and efficiency.

3. Features

DFS:

Depth-First Search (DFS) is an uninformed search algorithm that explores as far as possible along each branch before backtracking. A tree-based search algorithm has been implemented, with the inclusion of a visited set to prevent re-expansions of already explored nodes, therefore avoiding infinite loops due to cycles. The frontier uses a LIFO (Last in First Out) stack, expanding the most recently added node first. When multiple unvisited nodes are available, they are added to the frontier in descending order, so the smallest node is popped first, adhering to the ascending order requirements.

BFS:

Breadth First Search (BFS) is an uninformed search algorithm that explores horizontally before moving down to the next row. Breadth First Search uses FIFO (First in First out), which expands nodes based on the order it was added into the frontier. When more than one unvisited node is available, it is added in the order they are discovered. BFS also includes a visited set that is used to keep track of already explored nodes to prevent re-expansion which allows the algorithm to avoid infinite loops.

GBFS:

Greedy Best First Search is an informed algorithm that uses a heuristic function to determine which node to explore next from the current state's neighbours. It uses $f(n) = h(n)$ (cost to goal node from the current node) via straight line distance and sorts the frontier in increasing order to take the node with shortest distance. For this implementation, the GBFS is equipped with visited filter to ensure completeness in all cases.

A Star:

A* (A-Star) is an informed search algorithm that finds the lowest-cost path from an origin node to a destination node in a weighted graph. It evaluates each node using the function f(n) = g(n) + h(n), where g(n) is the actual cost from the origin to the current node, and h(n) is a heuristic estimate of the remaining cost to the goal. In this assignment, the straight-line (Euclidean) distance to the nearest destination. By combining both values, A* balances exploration and efficiency, always expanding the most promising node first. If the heuristic is admissible (never overestimates the true cost), A* is guaranteed to find the optimal solution, making it one of the most widely used and reliable pathfinding algorithms in AI.

CUS1 – Depth Limited Search:

Depth-Limited Search (DLS) is an uninformed search algorithm based on Depth-First Search (DFS) that explores a search tree only up to a specified depth limit. Traditional DFS continues expanding deeper into the tree until it either reaches the end of a branch or finds the goal, before exploring other branches. In contrast, DLS restricts this expansion by stopping once the predefined depth limit is reached. The choice of depth limit depends on the nature of the problem and is important in determining whether the algorithm can successfully find the goal. If the limit is set too shallow, the search may be cutoff before reaching the goal. Alternatively, if the limit is set too high, the search may not result in optimal path as it will keep

exploring deeper. Therefore, selecting and testing an appropriate depth limit is important to balance completeness and efficiency.

CUS2 - ALT:

ALT is an informed search algorithm that expands on A* by replacing the Euclidean distance heuristic with landmark-based lower bounds that are derived from Triangle inequality. This involves using multiple processes to select landmark nodes and compute and store shortest-path distances from all nodes to each landmark. When the code is running, for any node *v* and destination *t*, the heuristic value is calculated as the maximum of all distances between the landmark's distance to *v* and its distance to *t* |d(v, L) – d(t, L)|. The Triangle inequality guarantees to never overestimate the true distance, making the heuristic admissible, and taking the maximum ensures the tightest possible bound. Landmarks are selected randomly, and equal f(n) values follow the ascending node order as required.

Bugs:

The final search.py and all algorithms have no bugs present. The bugs found during testing were all fixed accordingly.

4. Testing:

The final search.py was tested thoroughly with specifically designed test cases that covers different scenarios. It includes general algorithm correctness tests to examine whether the algorithms are performing as expected, algorithm specific rule-based test cases and testing under unexpected scenarios such as goal is unreachable. Most test cases were built manually with expected results calculated. Some test cases were created with a help of AI tools with thorough evaluation, especially the complex test T15. Each test case's result is evaluated by path, count of nodes created and reached goal node to determine whether it failed or passed. The report details each test cases below.

T1: General correctness test case (Input: t1_gen_correct.txt)

- **Objective**: The aim of this test case is to evaluate the general correctness of the implemented search algorithms using a different input file from the one provided in the assignment. Each algorithm is expected to produce a result consistent with its search behaviour.
- **Output:** The algorithms all resulted as expected with valid paths according to their search strategies. BFS traversing horizontally and DFS going deeper through the branch. GBFS, AS, and ALT following the heuristic-guided route.

T2: Origin is the goal(Input: t2_origin_goal.txt)

- **Objective:** The test case scenario where origin is the goal node. All search algorithms must correctly identify the origin as a destination and return the origin alone as the solution path.
- **Output:** All algorithms passed the test with output of goal: 1, nodes_created: 1, path: 1.

T3: No Origin(Input: t3_no_origin.txt)

- **Objective:** Verify that all search algorithms correctly handle a missing origin node. The expected result is no goal found, 0 nodes created, and no path returned.
- **Initial output**: All algorithms returned 1 node created because nodes_created was initialised to 1 and no guard was in place to check whether the origin exists in the graph.

- **Fix:** Added an input guard at the start of each algorithm that checks whether origin and destinations are provided and whether origin exists in the nodes dictionary. Initialised nodes_created to 0, with the root node being counted only after it is created.
- **Fixed output:** All algorithms correctly return no goal, 0 nodes created, and no path when the origin is missing from the graph.

T4: Goal unreachable (Input: t4_goal_unreachable.txt)

- **Objective**: Testing scenario where the goal node exists in the graph but cannot be reached from the origin. The expected result is no goal found, the number of nodes created during the search, and no path returned.
- **Output:** Each algorithm iterated the entire reachable frontier from the origin without finding the goal, then correctly returned None for the goal and path.

T5: Empty Graph (Input: t5_empty.txt)

- **Objective:** All search algorithms must correctly handle an empty graph where no nodes or edges are defined. The expected result is no goal found, 0 nodes created, and no path returned.
- **Output:** All passed.

T6: A* - Cheap Path Over Direct-Looking Path (Input: t6_astar.txt)

- **Objective**: This test verifies that A* correctly selects the lowest-cost path rather than the geometrically shortest-looking one. Node 2 is directly in line with the goal (node 4) and has a short h(n), making it appear attractive. However, the edge (2,4) carries a cost of 100, making the indirect path via node 3 far cheaper at a total cost of 12. A* must use g(n) + h(n) to avoid being misled.
- **Output**: goal=4, nodes created=5, path $1 \rightarrow 3 \rightarrow 4$ (cost 12). A* correctly avoided the expensive direct edge ($1 \rightarrow 2 \rightarrow 4$, cost 105) in favour of the cheaper two-hop path.

T7: A* - Multiple Destinations – Cheapest Goal Wins Over Closest (Input: t7_astar.txt)

- **Objective**: This test verifies A* behaviour with multiple destinations where the geometrically closest destination (node 5, distance = 1 unit away) is reachable cheaply via a chain of low-cost edges (cost 3). A* must reach node 4 first since its f(n) is lower despite it appearing further away.
- **Output:** goal=4, nodes created=5, path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (cost 3). A* correctly prioritised the cheaper destinations over the closer-looking one, confirming that g(n) + h(n) prevents the greedy mistake that GBFS would make.

T8: Tie Breaking – Ascending order(Input: t8_tie_break.txt)

- **Objective:** The test case's goal is to verify whether the algorithms break ties by expanding the nodes with lower ID first.
- **Output:** All implementations correctly broke the tie expanding the lower ID node "2" instead of "3".

T9: ALT - Isolated Landmark Node (Input: t9_ALT.txt)

- **Objective:** The aim of the test is to verify that when a landmark has no ongoing edges, dijkstra_calc produces a table containing only {3: 0}, and landmark_heuristic will correctly skip the landmark,

rather than throwing a KeyError. Landmark 1 must supply a valid heuristic h(1) = 4 and guide the search to the goal.

- **Output:** The isolated landmark was skipped without error. Landmark 1 provided a valid heuristic and the search was guided to the goal with no wasted expansions.

T10: ALT – Landmark behind the Goal (Input: t10_ALT.txt)

- **Objective:** The test case's goal is to verify that placing a landmark (node 5) beyond the goal (node 4) produces the tightest possible triangle inequality heuristic. At each step h(v) = |d(5, v) – d(5, 4)| equals the exact true remaining distance, meaning no node outside the optimal path is expanded.
- **Output:** The heuristic was exact at each node along the path, and only four nodes were created (1 → 2 → 3 → 4).

T11: ALT – Landmark is the Goal (Input: t11_ALT.txt)

- **Objective:** The goal of the test is to certify that when the landmark is the same node as the destination (node 4), d(L, goal) = 0 making h(v) = |d(4, v) – 0| = d(4, v), the exact true distance at each node. There is an additional expensive direct edge (1,4): 20 must be created but never returned as a solution path.
- **Output:** The heuristic was exact throughout the expansion of nodes. The optimal path using nodes 2 and 3 was correctly found over the direct edge cost of 20.

T12: ALT – Max Over Landmarks is Taken (Input: t12_ALT.txt)

- **Objective:** The goal of this test is to verify that landmark_heuristic takes the maximum across all landmarks. Landmark 3 gives h(1) = 2 (a weak heuristic), while landmark 5 gives h(1) = 9 (exact). The correct implementation must return max(2,6) = 6. If min were mistakenly returned instead, the weaker heuristic would still find the correct path in this graph but would not meet the aggregation logic required.
- **Output:** The maximum was taken across both landmarks, giving h(1) = 6. The optimal path was found and the direct edge of cost 20 was discarded.

T13: ALT – Heuristic Never Overestimates (Input: t13_ALT.txt)

- **Objective:** This test's aim is to check landmark_heuristic satisfies admissibility h(v) <= true_cost(v, goal) at every node. Landmark 3 is in the middle of the graph, where triangle inequality is best exercised. The values are: h(1) = 3 <= 5, h(2) = 2 <= 6, h(4) = 2 <= 2, h(5) = 0 <= 0. If the heuristic is overestimated, then the optimal path 1 → 3 → 4 → 5 would be skipped.
- **Output:** The heuristic remained admissible at each node and the optimal path was explored.

T14: Infinite loop handling with visited (Input: t14_loop.txt)

- **Objective:** This test case is checking whether all algorithms handle fully bidirected nodes that cause infinite loops. It is expected that all handle the looping by tracking visited nodes and returning a valid path, goal reached.
- **Output:** All searches has resulted in pass with no algorithms stuck in a loop. For this test case DFS performed the worst by returning a path with all nodes " 1 → 2 → 3 → 4 → 5 → 6 → 7", while all informed algorithm along with BFS returned the optimal path of "1→ 4 → 7".

T15: Large graph stress test (Input: t15_large_graph.txt)

- **Objective:** The aim of this test case is to evaluate all algorithms on a large 40-node graph with two destination nodes (39 and 40), high branching factor, and a horizontal dead-end chain designed to stress uninformed search.
- **Output:** The algorithms all resulted as expected. BFS and DFS both found node 39 but via different paths, with DFS producing a longer path due to exploring the dead-end bottom row first. DLS with a limit of 5 returned a cutoff while a limit of 10 succeeded. GBFS, AS, and ALT all followed the heuristic-guided route to node 39 with GBFS creating the fewest nodes.

5. Insights:

Table 1 presents a theoretical comparison of all six implemented search algorithms using four notable properties. Time Complexity is defined as the measure of the amount of time taken by an algorithm to run as the function of the size of the input. Space Complexity is specified as the amount of memory required by the algorithm to solve a given problem [GeeksForGeeks, 2025]. For Optimality and Completeness, it is referring to if the algorithm can find the least-cost solution and it can always find an existing solution, respectively. Uninformed algorithms (DFS, BFS, DLS) offer completeness but no optimality, while the informed algorithms (GBFS, A*, ALT) incorporate heuristics as guidance.

Table 1: Comparison of Search Algorithm Complexity, Optimality and Completeness

Algorithm	Time Complexity	Space Complexity	Optimality	Completeness
DFS	$O(b^m)$	$O(b * m)$	No	Yes (with visited set)
BFS	$O(b^{d+1})$	$O(b^{d+1})$	No	Yes (if b is finite)
DLS	$O(b^l)$	$O(b * l)$	No	Yes (if limit is set correctly)
GBFS	$O(b^m)$	$O(b^m)$	No	Yes (with visited set)
A*	$O(b^{md})$	$O(b^d)$	Yes – heuristic is admissible - h(n) consistent for node n and successor n’	Yes (if b is finite)
ALT	Preprocessing: O(k(e + n log n) Querying: v(θ (|k|))	O(n * k)	Yes – guaranteed on static graphs with non-negative edge weights.	Yes (if graph is finite)

b = branching factor, d = path length (depth), m = max depth, l = depth limit, n = number of vertices on the graph, v = visited nodes, k = number of landmarks, e = number of edges

The route-finding problem prioritises optimality by finding the lowest cost path between the origin and destination. As a result, DFS, BFS, DLS, and GBFS may not always produce suitable solutions for this problem, as they are usually hit or miss on guaranteeing the lowest cost path. Therefore, a closer examination of A* and ALT is required, a claim that will be finalised in the conclusion. Although GBFS is an informed algorithm, it selects nodes purely based on the estimated distance to destination, completely

ignoring path cost. This is demonstrated directly by T6 and T7. In T6, GBFS reached the destination node 4 via the path 1 → 2 → 4, resulting in a path cost of 105, while the A* path followed 1 → 3 → 4, incurring a path cost of 12. Node 2 was geometrically closer than node 3, so GBFS expanded it, despite it being the most expensive direct edge. In T7, GBFS reached destination node 5 via the path 1 → 5, creating only three nodes, while A* reached the destination node 4 via 1 → 2 → 3 → 4, creating five nodes. GBFS found a valid path in less created nodes, however A* identified destination node 4 was cheaper overall.

DFS and DLS also do not account for edge costs, making them unsuited for weighted graphs. In T1, DFS created nine nodes and returned the path 1 → 2 → 3 → 4 → 5, while all the informed algorithms found node 5 via 1 → 3 → 5, creating five nodes. DFS had explored further than necessary because its uninformed depth-first strategy does not allow it to prioritise nodes closer to the goal or with lower path cost. DLS addresses this issue by adding a depth limit. However, if the limit is set too low, it will miss the destination node entirely. Even if the limit is set correctly the result does not guarantee to be the optimal path. Neither algorithm is appropriate when considering the lowest cost path. Similarly to DFS and DLS, BFS does not account for edge cost, but it also is unsuited for weighted graphs as it expands as many nodes as possible before it reaches its destination.

Both A* and ALT are considered optimal and utilise f(n) = g(n) + h(n) to guide the search, but they compute h(n) differently. A* uses Euclidean straight-line distance, which requires no preprocessing but consistently underestimates the true path cost on weighted directed graphs. ALT replaces the Euclidean heuristic with a landmark-based heuristic that produces a value much closer to the true remaining cost, at the expense of a preprocessing phase, increasing the time and space required to build and store the landmark table. For a single query on a small graph this overhead is unjustified, but for repeated queries on a large weighted directed graph, the heuristic results in fewer nodes created, making the upfront cost worthwhile.

From all tests, each algorithm produced outputs that aligned with its expected behaviour. Specifically for the informed algorithms on T1; GBFS, A*, and ALT all created five nodes and found the same path 1 → 3 → 5. This suggests that when the geometrically closest path happens to also be the cheapest, GBFS can produce the same result as A* and ALT. However, this is coincidental rather than guaranteed, as demonstrated by T6 and T7. On graphs where the geometrically closest path was not the cheapest, GBFS gives a suboptimal result while A* and ALT correctly identify the lowest cost path.

Using the theoretical complexities to predict behaviour when the graph is scaled, consider a graph with forty nodes, with a branching factor of four, a solution depth of six, and six landmarks for ALT. A* would have a space complexity of $O\ (b^d)$ = O (4^6) = 4,096 nodes that are stored in the memory, which will grow exponentially with the solution depth. ALT would have a space complexity of O (n * k) = O (40 * 6) = 240, which grows linearly with the number of nodes and landmarks, making it easier to manage at scale. At query, ALT would perform v(θ (|k|)) operations, making the query cost 50 * 6 = 300 compared to A*'s potential thousands. This behaviour validates the argument that ALT is the most suitable algorithm for the route-finding problem, particularly as the graph scales. The preprocessing cost is a one-time investment that pays off for multiple queries, and the tighter heuristic reduces the number of nodes visited compared to A*.

Table 2: T15: Large graph results with path cost

Algorithm	Nodes created	Goal Reached	Path	Path Cost
DFS	44	39	1 → 2→ 3 → 4 → 5 → 6 → 7 → 13 → 12→18 → 24 → 30 → 31 → 32 → 33 → 36 → 38 → 39	40

BFS	40	39	1→ 8 → 14 → 21 → 28 → 34 → 37 → 39	20
DLS(L=7)	274	39	1→ 8 → 14 → 21 → 28 → 34 → 37 → 39	20
GBFS	13	39	1→ 8 → 14 → 21 → 28 → 34 → 37 → 39	20
A*	24	39	1→ 8 → 14 → 21 → 28 → 34 → 37 → 39	20
ALT	21	39	1→ 8 → 14 → 21 → 28 → 34 → 37 → 39	20

This theoretical prediction is validated by the results of T15, a 40-node graph with a solution depth of seven. A* created 24 nodes while ALT created 21, confirming the landmark heuristic produces a tighter estimate than straight-line distance, resulting in fewer nodes expanded. Both algorithms found cost-optimal paths with a total path cost of 20. In contrast, DFS created 44 nodes and returned a path cost of 40 demonstrating that uninformed search on weighted graphs can find valid but highly suboptimal solutions. BFS and DLS also returned cost effective paths of 20 but at the expense of 40 and 274 nodes created respectively. DLS's high count is due to its recursive backtracking with visited removal. GBFS remained the most efficient at 13 nodes created, which is due to edge costs in the test case being similar. This is not always the case in real-life scenarios. The results confirm that A* and ALT offer the best balance between nodes created and path cost optimality, with ALT holding an advantage as the graph scales.

6. Research:

ALT:

The ALT algorithm was primarily sourced from Goldberg, A.V. & Harrelson, C (2005), *Computing the Shortest Path: A* Search Meets Graph Theory*, published in proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms. ALT was introduced as a preprocessing-based enhancement to A* search, motivated by the real-world problem of shortest path queries on large road networks. Additional theoretical context regarding landmark selection techniques and computing lower bounds, was drawn from Campbell Jr., N. (2018), *Using Quadrilaterals to Compute the Shortest Path* (arXiv:1603.00963v1). It provides formal proofs of the triangle inequality heuristic. The JGraphT Python library was also consulted to understand the background behind the A* extension and how ALT's parameters are structured [Michail, 2020].

ALT operates in two phases, a preprocessing phase that selects landmark nodes and runs Dijkstra's algorithm from each to build a landmark distance table. The other is the query phase that runs A* with h(n) replaced by the landmark-based heuristic:

h(v) = max over all landmarks k of | d (v, k) -d (t, k) |

with v representing the current node (same as n).

Standard A* utilises Euclidean straight-line distance as its heuristic, which is admissible but often underestimates the true path cost, particularly on graphs with directed edges. When h(n) underestimates, A* is comparable to uninformed search algorithms, as any node is as promising as another. ALT's landmark heuristic is computed from shortest-path distances within the graph, which naturally accounts for the directed edges and path costs. The value produced is much closer to the true remaining distance, so A* can deprioritise nodes that are further from the goal, thereby expanding fewer nodes.

GBFS selects node purely based on how close they appear to be to the destination, having no consideration for the edge cost incurred to reach it. So GBFS may traverse an expensive path simply because it has the

shortest straight-line distance. ALT, like A*, prevents this by always factoring in g(n), the actual edge cost so far, alongside the heuristic estimate.

The more meaningful distinction is between ALT and A*. Both algorithms share the same structure, priority queue, f(n) = g(n) + h(n) evaluation, and optimality, however, there is different in the representation of h(n). In ALT, h(v) is derived from real distances computed inside the graph itself. A node that is geometrically close to the destination but has a high-cost path or one-directional edges will receive a high h(n), whereas A* would treat it as promising.

The implementation of ALT was split over four functions, covering the two phases; landmark_heuristic, create_landmark_table, dijkstra_calc, and a_landmark_triangle_inequality_search. In the prepocessing phase, create_landmark_table selects landmark nodes and calls dijkstra_calc from each one. Dijkstra's algorithm using dijkstra_calc is run from a given landmark, returning a dictionary of every reachable node to its shortest distance from that landmark. These dictionaries are stored in LM_Table, where each landmark node ID (L) maps to its corresponding shortest distance dictionary returned by dijkstra_calc. By default, landmarks are chosen randomly using random.sample, where the number of landmarks *k* is determined by the size of the graph. A minimum of two and a maximum of sixteen are enforced. If landmarks are provided explicitly via the input file, random selection is skipped entirely.

In the query phase, a_landmark_triangle_inequality_search runs A* using landmark_heuristic instead of the standard Euclidean heuristic. For a given state, landmark_heuristic iterates over all landmarks and computes |d (k, state) – d (k, goal)| for each, taking the maximum output across all landmarks, and the minimum of all destinations. Nodes that are missing from the landmark's distance table, which may occur due to isolated landmarks or unreachable nodes on directed graphs, are skipped gracefully rather than throwing an error. In this case, the heuristic will fall back to 0 if no landmark can contribute a useful bound.

The search itself follows the structure of A*, with a min-heap ordered by f(n) = g(n) + h(n), a best_g dictionary to track the cheapest known cost to each node, and a stale-entry to skip nodes that have since been reached with less cost. The only structural difference from the standard A* implementation is the substitution of landmark_heuristic, replacing heuristic_distance.

7. Conclusion

In this paper six search algorithms were implemented, these algorithms include Breadth-First search (BFS), Depth-First search (DFS), Greedy-Best-First search (GBFS), A*, Depth-Limited Search (DLS) and A* Landmark Triangle inequality search algorithm (ALT). To find the best weighted directed tree-based search algorithm the six algorithms were analysed based on their path and path cost.

From the analysis, uninformed algorithms (BFS, DFS, DLS) were found to be unsuitable for weighted directed graphs. BFS was not an optimal algorithm as it does not evaluate edge cost during its search, and it expands unnecessary nodes when trying to reach its destination. DFS explores deeper branches with no cost considerations, where it returns paths with the highest cost most of the time. While, DLS has an improvement on DFS by limiting the depth, but it can be sensitive to the limit values resulting in cutoffs or producing similar results as DFS. GBFS, while informed, was also found to be unsuitable as it only considers the estimated distance to the goal and ignores actual path cost.

This leaves the most suitable algorithm being between A* and ALT. According to the analysis both A* and ALT excel at finding the shortest path, but differ in their denomination of the heuristic value, resulting in a large difference space and time complexity. A* has a space complexity which grows exponentially compared to ALT which has its space complexity that grows linearly. For time complexity ALT has one time preprocessing for its landmark table. In cases of small graphs, A* will provide the shortest path for lesser time and space complexity than ALT but in the case of a large graph ALT would be more efficient in finding the shortest path due to its linearly growing space complexity as shown with table 2.

An optimisation that can be done to improve ALT is selecting better landmarks. This can be done using planar landmark selection and farthest landmark selection. Planar landmark selection has landmarks selected by creating sectors in the graph and picking the furthest node in each sector as a landmark, to avoid having two landmarks next to each other, ALT will not process nodes in sector 2 that is close to the border between the sectors if the landmark for sector 1 is close to that border. Farthest landmark selection decides on a landmark by using a starting node and finding the node furthest away from it and select it as a landmark. This process is iterated, and in each iteration a node that is furthest away from the current set of landmarks is selected. This process is done to maximize the distance between a pair of nodes. [Goldberg & Harrelson, 2005]. Accuracy for the algorithm can be increased using these selected landmarks as it would give a better estimation for the true distance between the node and the destination

Acknowledgements/Resources

The following resources were used when implementing the search algorithms and in preparation of this report. Each entry includes a description of how it contributed to the final implementation and report.

1. Aitchison (2012), Apress, https://link.springer.com/chapter/10.1007/978-1-4302-3492-0_14 – Used to provide background information on route finding in the introduction.

2. aimacode/aima-python. (2022, May 11). *aima-python*. GitHub. https://github.com/aimacode/aima-python - used as a reference for the implementation of search.py and algorithms

3. ChatGPT: used for creating the large graph test case (T15)

4. GeeksForGeeks (2025), https://www.geeksforgeeks.org/dsa/shortest-path-algorithms-a-complete-guide/ - Used to provide background on the selected algorithms in the introduction, and for the implementation of DFS.

5. GeeksForGeeks (2026), https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/ - Used for describing the structure of the graph, as well as links to example BFS and DFS implementations. Utilised when creating DFS.

6. GeeksForGeeks (2025) , https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-dijkstras-algorithm/ - Used to understand the time complexity of Dijkstra's algorithm.

7. GeeksForGeeks (2025), https://www.geeksforgeeks.org/dsa/time-complexity-and-space-complexity/ - Provided definitions of space and time complexity for the insights section.

8. Goldberg & Harrelson (2005), https://www.cs.princeton.edu/courses/archive/spr06/cos423/Handouts/GH05.pdf – Primary source utilised in the research and implementation of ALT.

9. Campbell Jr. (2016), https://arxiv.org/pdf/1603.00963 – Used to define ALT and provide landmark-based lower bounds.

10. Michail (2020), https://python-jgrapht.readthedocs.io/en/jgrapht-1.5.0.3/api/algorithms/shortestpaths.html – Provided details on the parameters for ALT, as well as the citation used in text for Goldberg & Harrelson.

11. GeeksForGeeks, https://www.geeksforgeeks.org/artificial-intelligence/depth-limited-search-for-ai/ - Used to understand the concept of the Depth Limited Search Algorithm.

12. Week 3 Tutorial Practical Code (W3 search.py) - Used for Depth Limited Search reference code.

13. Claude AI - Used to generate A* and DLS Test cases.

References

Aitchison, A. (2012). *Pro Spatial with SQL Server 2012. Apress.* https://link.springer.com/chapter/10.1007/978-1-4302-3492-0_14

Campbell Jr., N (2016). *Using Quadrilaterals to Compute the Shortest Path.* (arXiv:1603.00963v1). https://arxiv.org/pdf/1603.00963

GeeksForGeeks. (2025). *Shortest Path Algorithms: A Complete Guide.* https://www.geeksforgeeks.org/dsa/shortest-path-algorithms-a-complete-guide/

GeeksForGeeks. (2026). *Graph Data Structure and Algorithms.* https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/

GeeksForGeeks. (2025). *Time and Space Complexity.* https://www.geeksforgeeks.org/dsa/time-complexity-and-space-complexity/

GeeksForGeeks. (2025). *Time and Space Complexity of Dijkstra's Algorithm*. https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-dijkstras-algorithm/

Goldberg, A.V. & Harrelson, C. (2005). Computing the Shortest Path: A* Meets Graph Theory. *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms.* https://www.cs.princeton.edu/courses/archive/spr06/cos423/Handouts/GH05.pdf

Michail, D (2020). *JGraphT Python Documentation: Shortest Paths.* https://python-jgrapht.readthedocs.io/en/jgrapht-1.5.0.3/api/algorithms/shortestpaths.html

Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: a Modern Approach* (4th ed.). Pearson.

GeeksForGeeks (2025). A* Search Algorithm. https://www.geeksforgeeks.org/dsa/a-search-algorithm/

Stanford CS Theory (1997 – updated through 2026). Introduction to A* (From Amit's Thoughts on Pathfinding). https://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html